

Individual Project Report

BCI Simulation Toolkit

Aahana Dahiya

LC2: Team 20

ENGR 13300: Transforming Ideas to Innovation – EPICS and VIP

Professor Aamod Samuel

December 12, 2025

1. Project Introduction

Brain-computer interfaces (BCIs) are developing technologies that allow for direct communication between the human brain and external devices. Many current generation BCI's rely on bulky headsets, laboratory-grade equipment, and invasive or semi-invasive procedures, which limits their accessibility to "laboratory or clinical environments, which are not conducive to the daily use of BCI devices" (J. Zhang et al). For my dream project, I would develop a non-invasive brain-computer interface, or BCI, to allow people to directly communicate with machines, or even with other people, through their thoughts. The purpose of my project is to build upon the underlying structures of my dream project and design a simplified, non-invasive simulation of a BCI system that can process and classify brain-like electrical activity in simulated EEG brainwave signals into meaningful categories such as imagined words or intentions. In order to do so, a K-Nearest Neighbors (KNN) model was chosen to be used, which is a supervised machine-learning model that is used to "classify unlabeled observations by assigning them to the class of the most similar labeled examples" (Z. Zhang). In exploring how these signals can be filtered, processed, and classified, this project aims to model the mechanisms behind real BCI systems in a more accessible and customizable environment.

This project is especially significant because BCI technologies have the potential to transform how people interact with technology, communicate, and perform tasks, particularly individuals with disabilities who face challenges with speech or movement and who may rely on alternative communication systems, and it is important for these technologies to have more intuitive and accessible systems. A system that can reliably interpret intended words or decisions from brain signals could offer users greater independence and new ways to interact with their environment, "thereby augmenting natural human abilities and presenting novel treatment and rehabilitation options for

individuals with neurological disorders” (H. Zhang et al.). Inspired by the capabilities that thought-driven interactions have regarding the expansion of human capability, I wanted to create a realistic version of that concept using tools available at our level of study, laying the groundwork for future innovations in similar technologies and interfaces. While my project does not measure real EEG activity, it simulates the pipeline of a functional BCI: generating signals, filtering noise, extracting features, and classifying patterns.

2. Project Overview of Inputs and Outputs

My project generates synthetic EEG-like signals by combining sine waves of varying frequencies and applying gaussian noise to mimic the variability of real brainwaves, utilizing several key inputs that allow users to control how these signals are generated, processed, and classified. Users can specify the desired dataset size, which strongly affects model and training stability, and the desired number of neighbors for the KNN model, which impacts model accuracy by controlling how many data points contribute to a final decision. They can also set the noise level to simulate realistic EEG interference as well as sampling rate and duration which are important for shaping the characteristics of the signals. For example, a user could input 300 samples per class, a noise level of 2, a sampling rate of 100, a duration of 2 seconds, and 3 k-neighbors, which would result in a higher accuracy model, and can be reliably used to classify signals with any of the three labels. Each input directly influences the behavior of the system, whether through the generation of synthetic signals or performance of the KNN model, which contributes to the overall output and performance, and can be evaluated to determine its accuracy and reliability in real-world applications.

Based on these inputs, the program produces several outputs that reflect the entire BCI simulation pipeline. These outputs include visualizations of the time-domain signals, frequency-domain plots generated using the Fast Fourier Transform (FFT), CSV files containing the information of signals and features extracted from those signals, and the predicted class labels and classification report from a KNN machine-learning model implemented through scikit-learn. After the report is printed, the user has a choice to either input a CSV file containing signal information to be tested or to generate another synthetic signal based on inputs indicating a class and noise level, resulting in a final prediction indicating whether the model correctly recognized the intended class. Overall, the inputs and outputs of the program help in the transformation of raw signal patterns into meaningful classifications, mirroring the functionalities of real BCI's.

3. User-defined Functions

In this project, the functionality of the BCI simulator are achieved through various user-defined functions (UDFs). Each function serves a specific role, from generating the synthetic EEG signals and adding noise, to extracting spectral features, training the classifier, and visualizing results for the user. These functions allow the program to remain organized, readable, and flexible, while allowing complex functions to be performed in a structured and reusable way.

The arguably most important function, `generate_synthetic_signal`, creates a synthetic sine-wave signal that simulates a simple brainwave component. It takes inputs for frequency, sampling rate, signal duration, amplitude, and phase. It calculates the total number of samples, constructs a time vector using `np.linspace`, and produces a sinusoidal waveform using the standard equation $\text{amp} * \sin(2\pi ft + \text{phase})$. The function outputs both the time vector and the generated signal, which acts as

the foundation for creating the signals by allowing the main program to generate base, harmonic, and drift components. Additionally, the inclusion of amplitude and phase inputs improves the flexibility and variability in the waveforms, which produces more realistic synthetic EEG data.

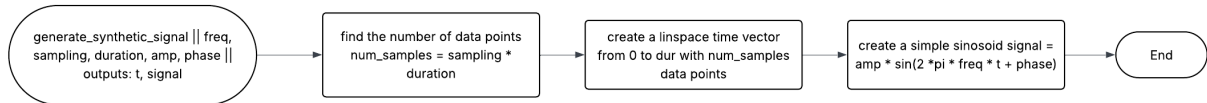


Figure 1: *generate_synthetic_signal* flowchart

The function `add_gaussian_noise` simulates environmental noise typically found in real EEG signals by adding Gaussian white noise to an existing signal created by the previous function `generate_synthetic_signal`. It takes the clean signal and a noise standard deviation, then generates random noise using `np.random.normal`, returning the noisy signal while maintaining the original signal's length and structure. This functionality is important in making the dataset more realistic with variability and improving the applicability of the model, ensuring that the classifier learns to distinguish patterns under real-world noisy conditions instead of memorizing unrealistic ideal waveforms.

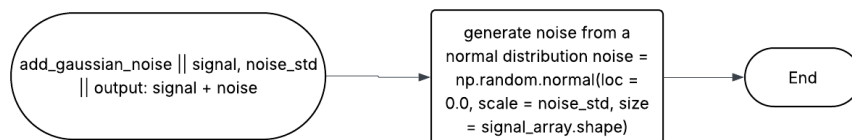


Figure 2: *add_gaussian_noise* flowchart

The function `save_signal_to_csv` saves a time-series signal to a CSV file. It takes a file path, a time vector, a signal vector, and a string for the header, then writes them row-by-row into a structured CSV file. It includes a unique error-handling mechanism using a `try/except` block that creates missing

folders parent directories using `Path.mkdir()`, if they do not exist already, and prevents the program from crashing if the save fails for any reason, returning either `True` or `False` to the main function, and allowing the other functionalities to work around the error. This function helps improve the transparency of the project by enabling users to save example EEG signals, and analyze their specific values.

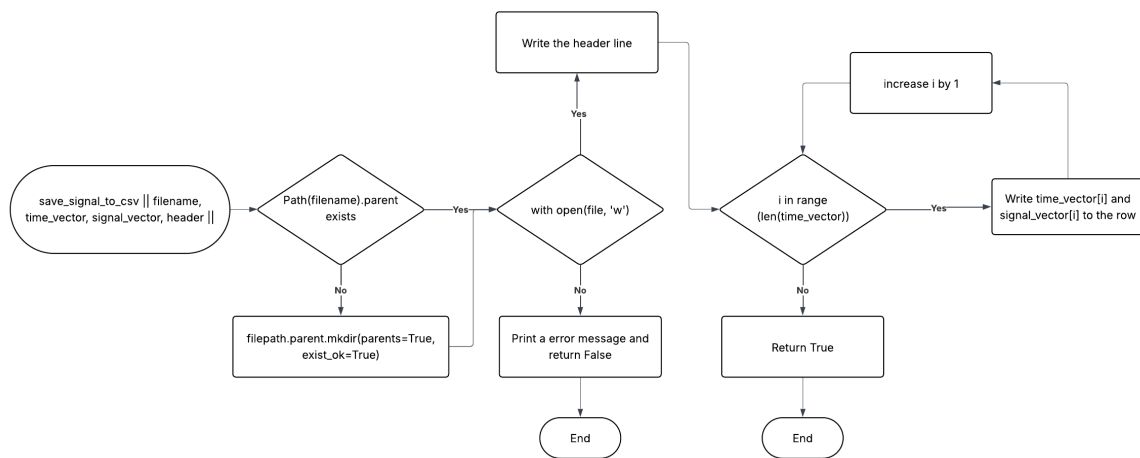


Figure 3: *save_signal_to_csv* flowchart

The function `save_features_csv` outputs the extracted EEG spectral features from the other functions along with their corresponding labels into a CSV file. It takes a filename, a matrix of feature vectors, and an array of class labels, writing one labeled feature vector per row with their respective labels. Similar to the previous save function, it uses `try/except` error-handling to prevent crashes and returns a Boolean success indicator to the main function. This function exports the entire dataset in a reusable form, allowing users to inspect or reuse feature data across sessions for testing or validation even after the program ends.

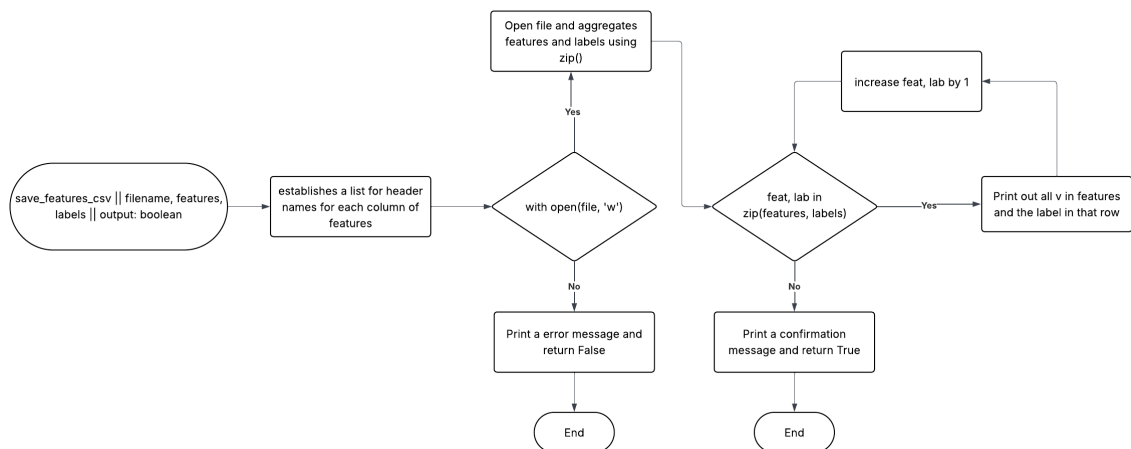


Figure 4: *save_features_csv* flowchart

The function `prompt_number` is responsible for handling the error-checking for all user inputs. It prompts the user for the required input inside a while True loop and converts the response into the desired type, namely int or float, for later calculations. This function continues to prompt users for the same input if they were to give invalid inputs, like letters rather than numbers, using a try/except ValueError, or were to provide inputs outside of the designated numeric bounds. This function is very important in ensuring that all user inputs for sampling rate, duration, noise, and other parameters are valid and safe for generating the synthetic signals, preventing any errors in the program.

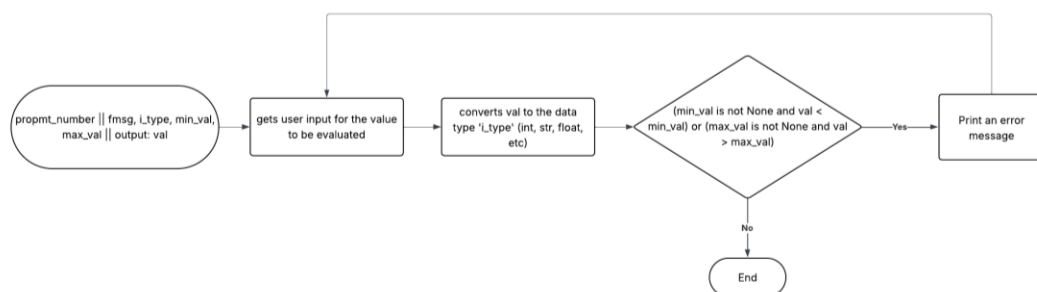


Figure 5: *prompt_number* flowchart

The function `synthesize_signal` constructs the full waveform based on a combination of signal components to create a richer composite signal using a main sinusoid at the label's base frequency, a smaller harmonic component at twice the frequency to simulate neuronal harmonics, and a slow drift signal representing baseline fluctuations in real EEG signals. It then combines all the components and adds Gaussian noise using the `add_gaussian_noise` function to produce a single synthetic EEG signal, mimicking multi-component brainwave structure and improving the realistic nature of the program.

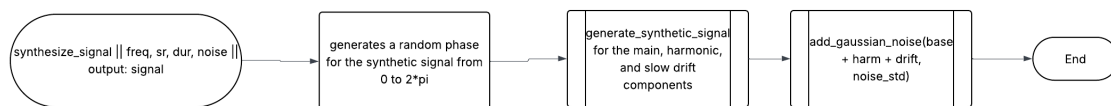
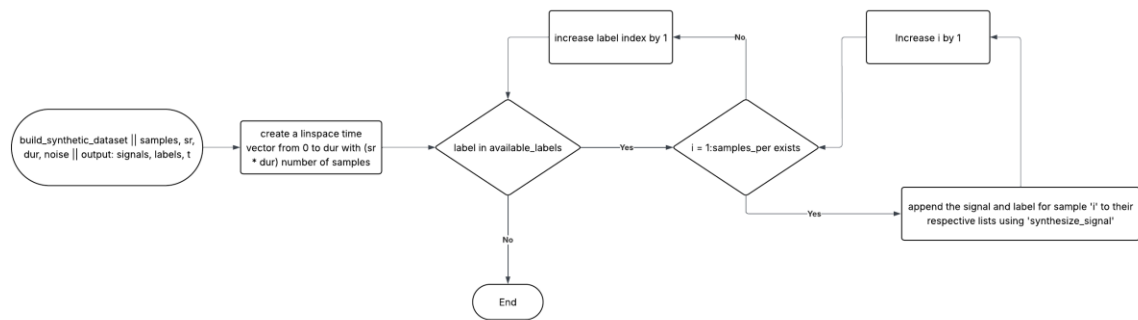
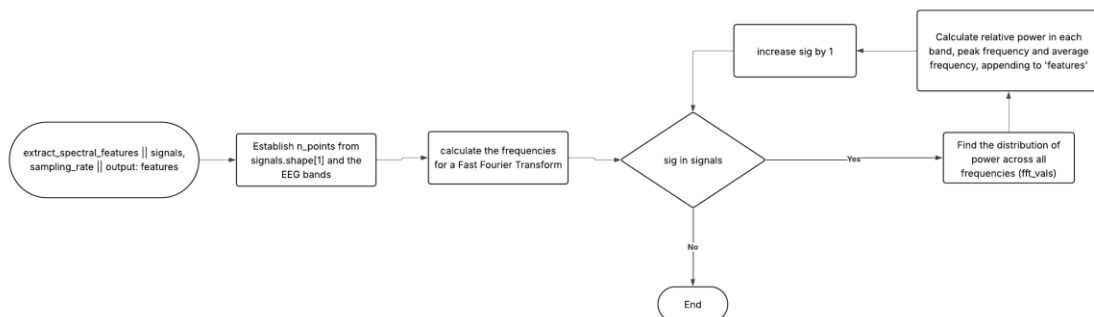


Figure 6: `synthesize_signal` flowchart

The function `build_synthetic_dataset` generates the full dataset used to train the classifier, looping over each label of “yes,” “no,” and “help”, producing the specified number of synthetic signals per label using the `synthesize_signal` function. It stores and returns all the generated signals and their corresponding labels in arrays as well as a time vector, allowing for streamlined feature and signal extraction as well as the KNN model training.

Figure 7: *build_synthetic_dataset* flowchart

The function `extract_spectral_features` computes the spectral features in the signals generated in `build_synthetic_dataset`, performing a Fast Fourier Transform (FFT) for each signal to obtain the power spectrum. It then uses that value to compute the relative power across the five EEG frequency bands, the peak frequency, and the spectral centroid, which is what the returned feature vector for each signal is composed of. This function is crucial in converting the signal data to be used by the KNN classifier to train and predict the labels.

Figure 8: *extract_spectral_features* flowchart

The function `train_classifier` trains a K-Nearest Neighbors classifier. It performs a train/test split using the scikit libraries on the inputted features and labels arrays to ensure the accuracy is validated

on unseen signals, scales features using StandardScaler, and trains the KNN model on the inputted number of k neighbors. It then evaluates the performance using accuracy and a full classification report, returning the precision, recall, F1-score and support values, encapsulating the entire machine-learning pipeline and ensuring that the project is transparent for user evaluation and easily adjustable regarding the hyperparameters.



Figure 9: train_classifier flowchart

The function plot_signal produces two types of visualization of an example signal, one being a time-domain plot and another being a frequency-domain magnitude spectrum. It calculates the FFT based on the inputted signal, generates appropriate frequency axes based on the inputted sampling rate, and displays the plots side-by-side. This function is crucial in helping users understand how each synthetic signal looks and how its spectral properties relate to its classification, enhancing the interpretability and transparency of the project.

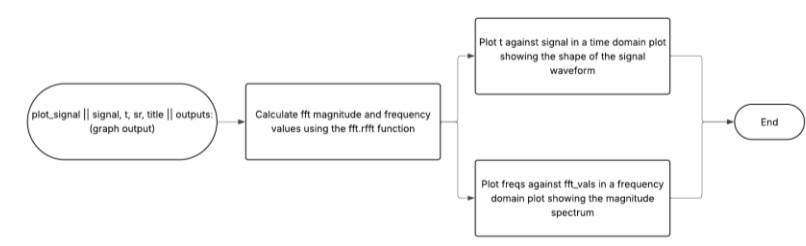


Figure 10: plot_signal flowchart

4. References

Zhang, Hongyu, et al. "Brain-Computer Interfaces: The Innovative Key to Unlocking Neurological Conditions." *International Journal of Surgery*, vol. 110, no. 9, 14 Aug. 2024, <https://doi.org/10.1097/js9.0000000000002022>.

Zhang, Jiayan, et al. "Recent Progress in Wearable Brain-Computer Interface (BCI) Devices Based on Electroencephalogram (EEG) for Medical Applications: A Review." *Health Data Science*, vol. 3, 1 Jan. 2023, <https://doi.org/10.34133/hds.0096>.

Zhang, Zhongheng. "Introduction to Machine Learning: K-Nearest Neighbors." *Annals of Translational Medicine*, vol. 4, no. 11, June 2016, pp. 218–218, <https://doi.org/10.21037/atm.2016.03.37>.

5. Appendix

1. User manual

There are many possible ways that a user can interact with the synthetic BCI model, in both the creation and export files of the generated EEG-like brainwave signals as well as in tested the validity and reliability of the developed KNN model. However, it should be noted that the output for the generated signals and features will likely differ in some capacity since new signals are generated on each run of the program and are not seeded values. Additionally, due to this difference, the accuracy scores may differ slightly between different runs and different generated datasets even with the same parameter inputs. Therefore, this should serve as a general guide to navigating the various aspects of the program and testing the different KNN models being trained and tested.

When the program is first executed, the title banner and first input prompt will appear after some loading time, asking for several parameters for the dataset and signal generation, namely the samples per class, which determines how many synthetic signals will be generated per class, noise standard deviation, which controls the level of Gaussian noise added to the signals, sampling rate, which indicates the number of data points recorded per second, the duration, which indicates the length of each signal, and K-Neighbors, which indicates how many data points influence the overall prediction. The selection of these parameters must be within the given boundaries in order for the model to run correctly, and failure to do so will result in an error message and the repetition of the same input until a valid response is received.

```
--==-- Synthetic BCI Classifier ==--  
Samples per class (5-500): 0  
Value must be between 5 and 500  
Samples per class (5-500): 600  
Value must be between 5 and 500  
Samples per class (5-500): 300  
Noise std (0-5): -1  
Value must be between 0 and 5  
Noise std (0-5): 6  
Value must be between 0 and 5  
Noise std (0-5): 3  
Sampling rate (32-2000): 30  
Value must be between 32 and 2000  
Sampling rate (32-2000): 40  
Duration (s) (0.1-10): 0  
Value must be between 0.1 and 10  
Duration (s) (0.1-10): 2  
K for KNN (1-31): 0  
Value must be between 1 and 31  
K for KNN (1-31): 3
```

Figure 11: Test for Error-checking of Input Bounds

If the user wishes for the training of a more accurate model, more samples per class, higher sampling rates, longer signal durations, and lower noise levels lead to clearer EEG-like waveforms with

more distinct frequency peaks, which allows the classifier to extract cleaner spectral features and improves overall accuracy. One such example would be an input of 300 samples per class, a noise level of 2, a sampling rate of 200, a duration of 1 seconds, and 3 K-Neighbors, a model will be created with around 0.956 overall accuracy.

```

===== Synthetic BCI Classifier =====
Samples per class (5-500): 300
Noise std (0-5): 2
Sampling rate (32-2000): 200
Duration (s) (0.1-10): 1
K for KNN (1-31): 3

Inspect example signal? (y/n) n
Number of example signals to save (0-Samples): 0

[main] Created 900 signals
[main] No signals saved.
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.967
Report:

```

	precision	recall	f1-score	support
help	0.98	0.95	0.97	60
no	0.94	0.98	0.96	60
yes	0.98	0.97	0.97	60
accuracy			0.97	180
macro avg	0.97	0.97	0.97	180
weighted avg	0.97	0.97	0.97	180

```

=====

```

Figure 12: Sample Inputs for Higher Accuracy KNN Model

In contrast, high noise levels, very short durations, low sampling rates, or too few samples per class blur the frequency information and produce unstable feature values, making it harder for the classifier to correctly separate the classes and lowering overall performance. One such example would be an input of 50 samples per class, a noise level of 4, a sampling rate of 40, a duration of 0.5 seconds, and 2 K-Neighbors, a model will be created with around 0.333 overall accuracy.

```

----- Synthetic BCI Classifier -----
Samples per class (5-500): 50
Noise std (0-5): 4
Sampling rate (32-2000): 40
Duration (s) (0.1-10): 0.5
K for KNN (1-31): 2

Inspect example signal? (y/n) n
Number of example signals to save (0-Samples): 0

[main] Created 150 signals
[main] No signals saved.
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.333
Report:

```

	precision	recall	f1-score	support
help	0.43	0.60	0.50	10
no	0.25	0.30	0.27	10
yes	0.25	0.10	0.14	10
accuracy			0.33	30
macro avg	0.31	0.33	0.31	30
weighted avg	0.31	0.33	0.31	30

```

-----

```

Figure 13: Sample Inputs for Lower Accuracy KNN Model

Additionally, the user will be able to select whether they wish for the prediction of a randomly selected image from the generated dataset with a “y” or “n”, along with its respective time-domain and frequency-domain graphs and its predicted label based on the model. A selection of “n”, essentially meaning “no”, will produce outputs like the outputs above in *Figure 12* and *Figure 13*, with no outputted graph, no sample true and predicted label, and an indication of the end of that section. A selection of “y”, essentially meaning “yes”, will randomly select one of the signals from the test dataset, unseen by the KNN model. The time-domain and frequency-domain graphs of that signal will be outputted to the user, as well as the true and predicted label for that signal. Using the same inputs for *Figure 12*, we can see that the model has correctly identified the “help” signal. It should be noted that the signal graphs will not be the same on each run, as the signals are being newly generated on each run of the program. Additionally, if using a lower accuracy model, it is possible for some runs to

correctly identify the test signal and for others to misclassify them even with the same parameters, since there is still some level of accuracy above 0%.

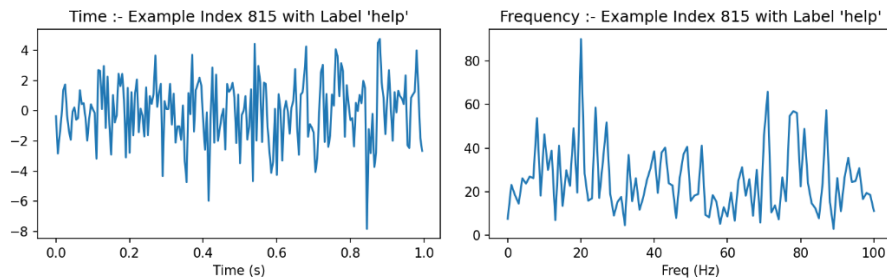


Figure 14: Time-domain and Frequency-Domain Graphs for Randomly Selected Test Signal

```

----- Synthetic BCI Classifier -----
Samples per class (5-500): 300
Noise std (0-5): 2
Sampling rate (32-2000): 200
Duration (s) (0.1-10): 1
K for KNN (1-31): 3

Inspect example signal? (y/n) y
Number of example signals to save (0-Samples): 0

[main] Created 900 signals
[main] No signals saved.
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.961
Report:

```

	precision	recall	f1-score	support
help	0.94	0.97	0.95	60
no	0.98	0.93	0.96	60
yes	0.97	0.98	0.98	60
accuracy			0.96	180
macro avg	0.96	0.96	0.96	180
weighted avg	0.96	0.96	0.96	180

```

True label: help, Predicted: help

-----

```

Figure 15: Output of Higher Accuracy Model with Label Prediction

The user will also be asked how many signals they wish to save for each label, with a given range of 0 up until the previously inputted value for the samples per class. For any input other than 0, the designated number of signals will be saved to a folder called “output_signals”. For example, if a user wishes to select “2” for the number of signals exported per class, 2 signals for each label will appear under that folder, with the time and signal data for that run. It should be noted that subsequent runs will not rewrite this folder, but will instead add more files to it, and care should be taken to organize these files as runs are conducted.

```

----- Synthetic BCI Classifier -----
Samples per class (5-500): 300
Noise std (0-5): 2
Sampling rate (32-2000): 200
Duration (s) (0.1-10): 1
K for KNN (1-31): 3

Inspect example signal? (y/n) n
Number of example signals to save (0-Samples): 2

[main] Created 900 signals
[main] Saved 2 signals per class to 'output_signals/'
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.972
Report:

```

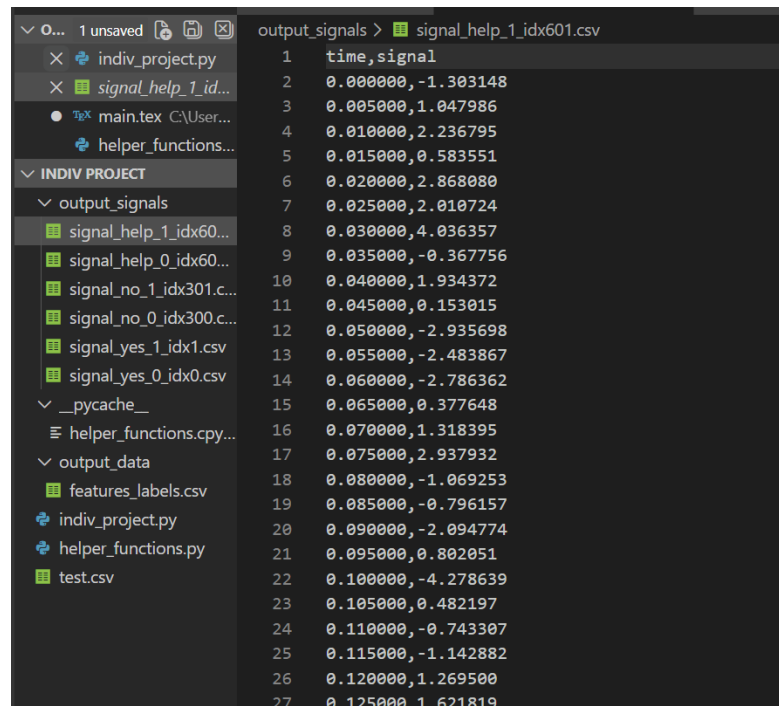
	precision	recall	f1-score	support
help	0.94	1.00	0.97	60
no	0.98	0.97	0.97	60
yes	1.00	0.95	0.97	60
accuracy			0.97	180
macro avg	0.97	0.97	0.97	180
weighted avg	0.97	0.97	0.97	180

```

-----

```

Figure 16: Terminal Output for Affirming Selection of File Export



	time	signal
1	0.000000	-1.303148
2	0.005000	1.047986
3	0.010000	2.236795
4	0.015000	0.583551
5	0.020000	2.868080
6	0.025000	2.010724
7	0.030000	4.036357
8	0.035000	-0.367756
9	0.040000	1.934372
10	0.045000	0.153015
11	0.050000	-2.935698
12	0.055000	-2.483867
13	0.060000	-2.786362
14	0.065000	0.377648
15	0.070000	1.318395
16	0.075000	2.937932
17	0.080000	-1.069253
18	0.085000	-0.796157
19	0.090000	-2.094774
20	0.095000	0.802051
21	0.100000	-4.278639
22	0.105000	0.482197
23	0.110000	-0.743307
24	0.115000	-1.142882
25	0.120000	1.269500
26	0.125000	1.621819

Figure 17: Example Data for Exported File Selection

The next section of the program is related to the testing of the generated model. This can either be done by a file input or a synthesized signal. The user can also choose the “quit” option to end the program. If the user were to select the “file” option, the user would then input the name of the CSV file with the data in it. The program will then plot the graphs for this signal and return the predicted label. It should be noted that the parameters for the dataset generation should remain the same, otherwise the model may not be able to accurately predict the label. One way to do this is by using the generated data from the exported CSV file from a previous run in the next run with the same parameters, since the data will be unseen for the newly generated model.

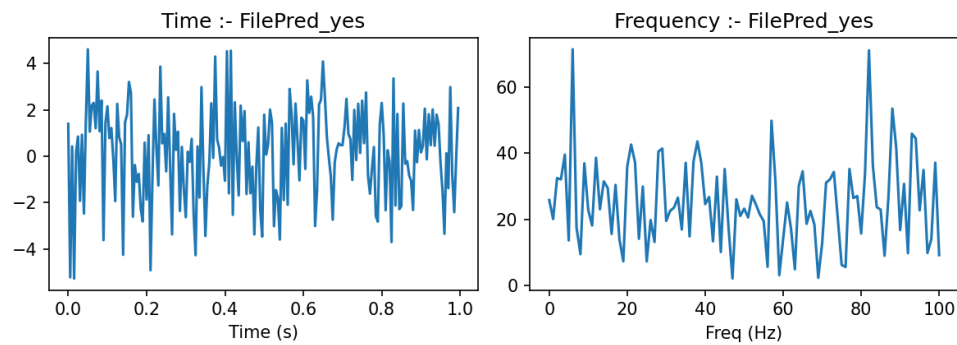


Figure 18: Graph Outputs from Imported File Data

```

===== Synthetic BCI Classifier =====
Samples per class (5-500): 300
Noise std (0-5): 2
Sampling rate (32-2000): 200
Duration (s) (0.1-10): 1
K for KNN (1-31): 3

Inspect example signal? (y/n) n
Number of example signals to save (0-Samples): 2

[main] Created 900 signals
[main] Saved 2 signals per class to 'output_signals/'
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.972
Report:

```

	precision	recall	f1-score	support
help	0.94	1.00	0.97	60
no	0.98	0.97	0.97	60
yes	1.00	0.95	0.97	60
accuracy			0.97	180
macro avg	0.97	0.97	0.97	180
weighted avg	0.97	0.97	0.97	180

```

=====
Interactive prediction mode. Type 'quit' to exit.

Type 'file', 'synth', or 'quit': file
Enter CSV path or 'cancel': test.csv
Predicted label: yes

Type 'file', 'synth', or 'quit': 

```

Figure 19: Terminal Output for "File" Option Selection

If a user were to select the “synth” option, the program would prompt the user for an input of the desired true label, either “yes”, “no”, “help”, or a random selection between the three, as well as the noise standard deviation for the test signal. The program will then use that to generate a new synthetic signal with the applied Gaussian noise, and will test it with the KNN model from before to

check the accuracy of the predictions. Using the same inputs from *Figure 12*, a selection of “yes” and “2” for the label and noise, the following will be outputted. It should be noted that the output may vary across different runs of the same inputs, as stated previously. *Figure 21* also shows the output for inputting “quit”, indicating the end of the program.

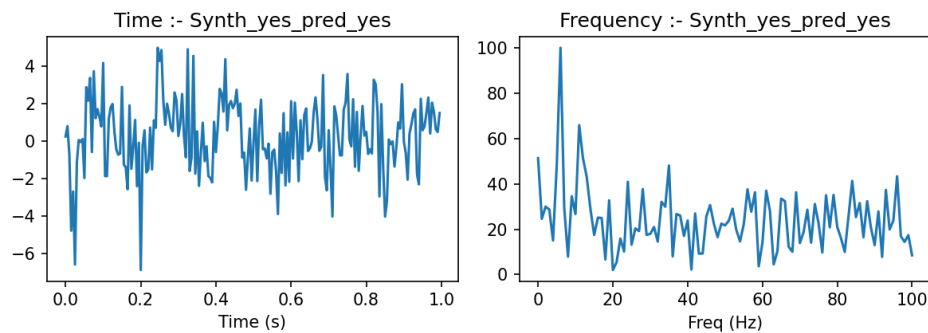


Figure 20: Synthesized "Yes" Signal for "Synth" Option

```

===== Synthetic BCI Classifier =====
Samples per class (5-500): 300
Noise std (0-5): 2
Sampling rate (32-2000): 200
Duration (s) (0.1-10): 1
K for KNN (1-31): 3

Inspect example signal? (y/n) n
Number of example signals to save (0-Samples): 0

[main] Created 900 signals
[main] No signals saved.
[save_features_csv] Saved features to output_data\features_labels.csv

Accuracy: 0.956
Report:

```

	precision	recall	f1-score	support
help	0.95	0.93	0.94	60
no	0.98	0.97	0.97	60
yes	0.94	0.97	0.95	60
accuracy			0.96	180
macro avg	0.96	0.96	0.96	180
weighted avg	0.96	0.96	0.96	180

```

=====
Interactive prediction mode. Type 'quit' to exit.

Type 'file', 'synth', or 'quit': synth
Label (yes, no, help) or 'random': yes
Noise std for test signal: 2
Synthesized true: yes, Predicted: yes

Type 'file', 'synth', or 'quit': quit

~~~~~Program completed~~~~~

```

Figure 21: Terminal Output for "Synth" Option

2. Flowchart

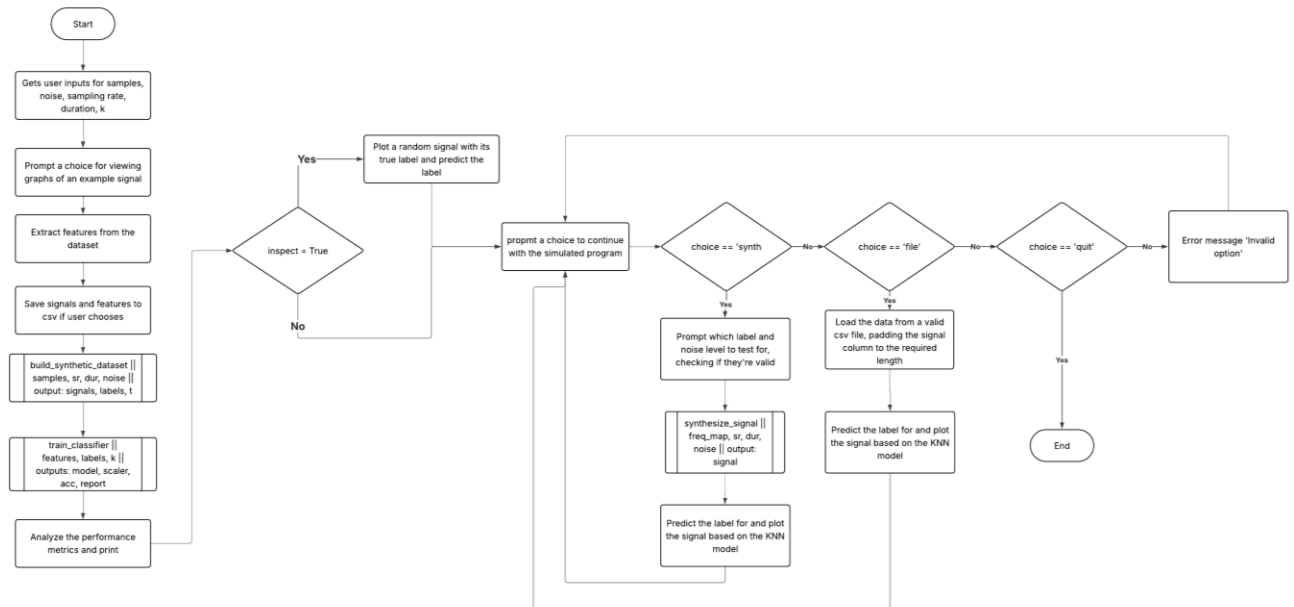


Figure 22: Flowchart of Entire BCI System

3. Code

```
#####
```

Course Number: ENGR 13300

Semester: Fall 2025

Description: Generates synthetic EEG-like signals for three classes ('yes', 'no', 'help'), extracts spectral features, trains a KNN classifier to predict the class labels, outputs the classification report and time and frequency domain graphs of the signal, and allows for the prediction of new synthetic or user-provided

signals, as well as the option to save signals and features to CSV files.

Assignment Information:

Assignment: Individual Project

Team ID: LC2 - 20

Author: Aahana Dahiya, dahiya1@purdue.edu

Date: 12/4/2025

Academic Integrity Statement:

I have not used source code obtained from any unauthorized source, either modified or unmodified; nor have I provided another student access to my code. The project I am submitting is my own original work.

"""

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import csv, os
```

```
from pathlib import Path
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import classification_report, accuracy_score
```

```
from helper_functions import generate_synthetic_signal, add_gaussian_noise,
save_signal_to_csv, save_features_csv

# global constants used throughout the program

DEFAULT_SAMPLING_RATE = 256

DEFAULT_DURATION = 1.0

AVAILABLE_LABELS = ['yes', 'no', 'help']

FREQ_MAP = {'yes':6.0,'no':10.0,'help':20.0}

# theta: yes/no

# alpha: higher cognitive loads indicate no

# beta: help (more alertness)


# directories for saving output data and signals

FEATURE_PATH = Path("output_data")

FEATURE_PATH.mkdir(exist_ok=True, parents=True)

SIGNAL_PATH = Path("output_signals")

SIGNAL_PATH.mkdir(exist_ok=True, parents=True)


def prompt_number(msg, i_type, min_val=None, max_val=None):

    while True: # infinite loop until valid input

        try:

            val = i_type(input(msg + " ")) # convert input to desired type

            # checks if within bounds
```

```
    if (min_val is not None and val < min_val) or (max_val is not None and val > max_val):
        print(f"Value must be between {min_val} and {max_val}")

    else:

        return val

except ValueError: # if conversion fails

    print("Invalid input. Try again.")

def synthesize_signal(base_freq, sampling_rate, duration, noise_std):

    # generates noisy synthetic EEG signal

    phase = np.random.uniform(0, 2*np.pi) # random phase for variability

    # main, harmonic, and slow drift components

    _, base = generate_synthetic_signal(base_freq, sampling_rate, duration, 1.0, phase) # core
signal

    _, harm = generate_synthetic_signal(base_freq*2, sampling_rate, duration, 0.3, phase/2) #
smaller harmonic like real EEG

    _, drift = generate_synthetic_signal(0.5, sampling_rate, duration, np.random.uniform(0,0.2), 0)

# slow drift component like baseline drift

    return add_gaussian_noise(base + harm + drift, noise_std) # add noise to combined signal

def build_synthetic_dataset(samples_per_class, sampling_rate, duration, noise_std):

    signals, labels = [], [] # lists to hold signals and labels

    t = np.linspace(0, duration, int(sampling_rate*duration), endpoint=False) # time vector

    # loops through each label and generates specified number of samples
```

```
for label in AVAILABLE_LABELS:

    for i in range(samples_per_class):

        # generate synthetic signal for current label

        signals.append(synthesize_signal(FREQ_MAP[label], sampling_rate, duration, noise_std))

        labels.append(label)

return np.array(signals), np.array(labels), t


def extract_spectral_features(signals, sampling_rate):

    n_points = signals.shape[1] # number of points in each signal

    freqs = np.fft.rfftfreq(n_points, 1/sampling_rate) # calculates the frequencies corresponding to
the output of a real Fast Fourier Transform

    bands = {'delta':(0.5,4),'theta':(4,8),'alpha':(8,13),'beta':(13,30),'gamma':(30,60)} # standard EEG
bands

    features = []

    for sig in signals:

        fft_vals = np.abs(np.fft.rfft(sig))**2 # power spectrum of the signal (distribution of power
across frequencies)

        total_power = np.sum(fft_vals)+1e-12 # avoid division by zero

        band_rel = [np.sum(fft_vals[(freqs>=lo)&(freqs<=hi))]/total_power for lo,hi in bands.values())

# relative power in each band

    peak_freq = freqs[np.argmax(fft_vals)] # frequency with maximum power
```



```
centroid = np.sum(freqs*fft_vals)/total_power # spectral centroid (weighted average
frequency)

features.append(band_rel + [peak_freq, centroid])

return np.array(features) # return feature matrix

def train_classifier(X, y, k=5):

    # imported function from sklearn to split dataset into training and testing sets

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y) # 20% for testing
and 80% for training, ensures correct class proportions

    scaler = StandardScaler() # standardize features to unit variance

    X_train_scaled, X_test_scaled = scaler.fit_transform(X_train), scaler.transform(X_test) #
applying scaling parameters from training set to test set

    model = KNeighborsClassifier(n_neighbors=k) # imported function to create KNN model

    model.fit(X_train_scaled, y_train) # train model on training data and labels

    preds = model.predict(X_test_scaled) # predict labels for test set

    return model, scaler, accuracy_score(y_test,preds), classification_report(y_test, preds,
zero_division=0)

def plot_signal(signal, t, sr, title="Signal"):

    # plots signal in time domain and magnitude spectrum
```

```
fft_vals = np.abs(np.fft.rfft(signal)) # fft magnitude values

freqs = np.fft.rfftfreq(len(signal), 1/sr) # corresponding frequencies


plt.figure(figsize=(8,3))

plt.subplot(1,2,1) # time domain plot showing the shape of the signal waveform

plt.plot(t, signal) # oscillations indicating predictable patterns for classification

plt.xlabel("Time (s)")

plt.title("Time :- " + title)


plt.subplot(1,2,2) # frequency domain plot showing magnitude spectrum

plt.plot(freqs, fft_vals) # peaks indicate dominant frequencies for classification + bands for
feature extraction

plt.xlabel("Freq (Hz)")

plt.title("Frequency :- " + title)

plt.tight_layout()

plt.show()


def main():

    print("--==-- Synthetic BCI Classifier ==--")

    # user inputs for dataset parameters

    samples = prompt_number("Samples per class (5-500):", int, 5, 500)

    noise = prompt_number("Noise std (0-5):", float, 0, 5)

    sr = prompt_number("Sampling rate (32-2000):", int, 32, 2000)
```

```
dur = prompt_number("Duration (s) (0.1-10):", float, 0.1, 10)

k = prompt_number("K for KNN (1-31):", int, 1, 31)

inspect = input("\nInspect example signal? (y/n) ").strip().lower() == 'y'

n_save = prompt_number("Number of example signals to save (0-Samples):", int, 0, samples)

# Build dataset

signals, labels, t = build_synthetic_dataset(samples, sr, dur, noise)

print(f"\n[main] Created {len(signals)} signals")

if n_save > 0:

    for label in AVAILABLE_LABELS: # Save n_save signals per class

        # find indices of signals with this label

        label_indices = np.where(labels == label)[0]

        for j, idx in enumerate(label_indices[:n_save]): # save first n_save signals of this label

            file_path = SIGNAL_PATH / f"signal_{label}_{j}_idx{idx}.csv"

            save_signal_to_csv(str(file_path), t, signals[idx])

        print(f"[main] Saved {n_save} signals per class to '{SIGNAL_PATH}'")

else:

    print("[main] No signals saved.")

features = extract_spectral_features(signals, sr) # extract features from all signals
```

```
save_features_csv(str(FEATURE_PATH/"features_labels.csv"), features, labels)

model, scaler, acc, report = train_classifier(features, labels, k) # train KNN classifier

# report contains performance metrics:

# Precision: accuracy of positive predictions

# recall: measures number of positive cases

# F1-score: weighted average of precision and recall

# support: number of actual occurrences of each class

print(f"\nAccuracy: {acc:.3f}\nReport:\n{report}") # print performance

if inspect: #check if user wants to inspect example signals

    idx = np.random.randint(len(signals)) # random index to inspect

    plot_signal(signals[idx], t, sr, title=f"Example Index {idx} with Label '{labels[idx]}'")

    feat = extract_spectral_features(signals[idx].reshape(1,-1), sr)

    pred = model.predict(scaler.transform(feat))[0] # predict label for this signal

    print(f"True label: {labels[idx]}, Predicted: {pred}\n")

print("-----")

print("Interactive prediction mode. Type 'quit' to exit.") # loop for user to test predictions

while True:
```

```
choice = input("\nType 'file', 'synth', or 'quit': ").strip().lower()

if choice=='quit': # exit the loop and end program
    break

elif choice=='synth': # synthesize new signal for prediction
    label = input(f"Label ({', '.join(AVAILABLE_LABELS)}) or 'random': ").strip().lower()
    if label=='random':
        label = np.random.choice(AVAILABLE_LABELS) # random label if user chooses
    else:
        label = label.lower()

    while label not in AVAILABLE_LABELS:
        label = input("Invalid label. Try again: ").strip().lower() # validate label input

    noise_lvl = prompt_number("Noise std for test signal:", float, 0, 5)
    sig = synthesize_signal(FREQ_MAP[label], sr, dur, noise_lvl) # generate signal
    feat = extract_spectral_features(sig.reshape(1,-1), sr)
    pred = model.predict(scaler.transform(feat))[0]
    print(f"Synthesized true: {label}, Predicted: {pred}") # print prediction
    plot_signal(sig, t, sr, title=f"Synth_{label}_pred_{pred}")

elif choice=='file': # load signal from user-provided CSV file
```

```
path = input("Enter CSV path or 'cancel': ").strip()

if path.lower()=='cancel': # user chose to cancel
    continue

try:
    data = np.loadtxt(path, delimiter=",", skiprows=1) # load CSV data skipping header
    if data.shape[1]>1: # get signal column
        sig = data[:,1]
    else:
        sig = data[:,0]

    if len(sig)<10: # too short to be valid
        print("Too short")
        continue

    if len(sig)<len(t): # pads to required length
        sig = np.pad(sig,(0,len(t)-len(sig)),'constant')
    else:
        sig = sig[:len(t)]

    feat = extract_spectral_features(sig.reshape(1,-1), sr)
    pred = model.predict(scaler.transform(feat))[0]
```

```
        print(f"Predicted label: {pred}") # print predicted label

        plot_signal(sig, t, sr, title=f"FilePred_{pred}")

    except Exception as e:

        print("Error reading file:", e)

    else:

        print("Invalid option.")

    print("\n~~~~~Program completed.~~~~~\n")

if __name__=="__main__":

    main()
```

```
"""

Course Number: ENGR 13300

Semester: Fall 2025


Description: This module provides helper functions for the Decoding Intent BCI Simulator Project
for creating and saving synthetic EEG-like signals.


Assignment Information:

Assignment: Individual Project
```

Team ID: LC2 - 20

Author: Aahana Dahiya, dahiya1@purdue.edu

Date: 12/4/2025

Academic Integrity Statement:

I have not used source code obtained from any unauthorized source, either modified or unmodified; nor have I provided another student access to my code. The project I am submitting is my own original work.

"""

```
import numpy as np
```

```
import csv
```

```
from pathlib import Path
```

```
def generate_synthetic_signal(freq, sampling, duration, amp=1.0, phase=0.0):
```

```
    # inclusion of optional inputs for amplitude and phase to allow more flexible signal generation
```

```
    num_samples = int(sampling * duration) # total number of data points
```

```
    t = np.linspace(0, duration, num_samples, endpoint = False) # generates time vector, avoid
```

```
double-counting endpoint
```

```
    signal = amp * np.sin(2 * np.pi * freq * t + phase) # create a simple sinusoid
```

```
    # Returns time vector and signal as numpy arrays
```

```
    return t, signal
```



```
def add_gaussian_noise(signal_array, noise_std): # generates noise from normal distribution to
model environmental interference

    noise = np.random.normal(loc = 0.0, scale = noise_std, size = signal_array.shape)

    #artificially corrupt a given dataset by adding random fluctuation

    return signal_array + noise


def save_signal_to_csv(filename, time_vector, signal_vector, header="time, signal"):

    try: # attempts to save the signal to a CSV file

        filepath = Path(filename)

        if not filepath.parent.exists(): # checks if given file exists, otherwise creates the parent
directories

            filepath.parent.mkdir(parents=True, exist_ok=True)

        with open(filepath, 'w', newline='') as csvfile:

            # saves the time and signal values in csv format

            writer = csv.writer(csvfile)

            writer.writerow(header.split(', ')) # writes the header row

            for i in range(len(time_vector)): # iterates through each data point

                t = time_vector[i]

                s = signal_vector[i]

                writer.writerow([f"{t:.6f}", f"{s:.6f}"])

            return True # indicates successful save
```

```
except Exception as e: # any error occurred during the save process

    print(f"[helper_functions.save_signal_to_csv] Error while saving file: {e}") # prints error
message

    # returns False to allow reattempt rather than crashing

    return False

def save_features_csv(filename, features, labels):

    header = ['delta','theta','alpha','beta','gamma','peak_freq','centroid','label'] # column names

    try: # attempts to save features and labels to CSV

        with open(filename, 'w', newline='') as f:

            f.write(','.join(header) + '\n')

            for feat, lab in zip(features, labels): # aggregates features and labels

                f.write(','.join(f"{v:.6f}" for v in feat) + f",{lab}\n") # writes each feature vector and label

    print(f"[save_features_csv] Saved features to {filename}")

    return True # indicates successful save

except Exception as e: # any error occurred during the save process

    print(f"[save_features_csv] Error: {e}")

    return False # indicates failure to save
```